CHAPTER 09

Databases Deep Dive

SQL vs NoSQL, the four NoSQL families, and the CAP theorem — how to choose the right data store for your service

SQL · NoSQL · ACID · BASE · CAP

Lesson 09 — Architecture Course

A G E N D A

What We'll Cover

01

The Relational Model

Tables, keys, ACID — why SQL has ruled for 50 years

02

Indexes

B-trees, scan vs seek, clustered vs non-clustered

03

What is NoSQL?

Schema-less data, BASE, and horizontal scaling

04

The 4 NoSQL Families

Document, Key-Value, Wide-Column, Graph

05

SQL vs NoSQL

Head-to-head comparison across 7 dimensions

06

The CAP Theorem

Consistency, Availability, Partition tolerance

07

Decision Guide

When to choose which — practical rules

0 1 — THE RELATIONAL MODEL

Tables, Keys & ACID

Customers

PK id	name	city
1	Dana	Holon
2	Avi	Haifa

1 : N

Orders

PK id	FK cust	total
10	1	₪250
11	2	₪90

A — Atomicity

All-or-nothing — a transaction fully commits or fully rolls back

C — Consistency

Every transaction moves the DB from one valid state to another

I — Isolation

Concurrent transactions don't see each other's partial work

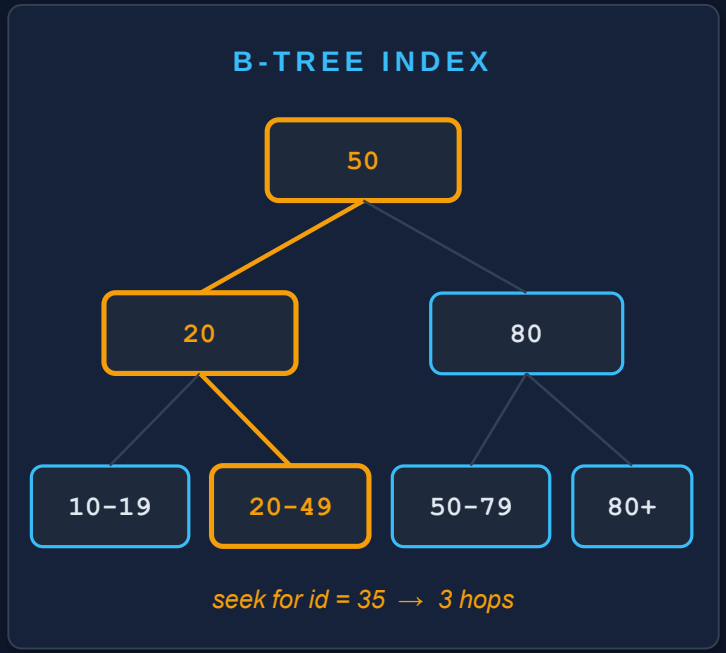
D — Durability

Once committed, data survives crashes — written to disk/WAL

Schema-on-write • Scales vertically • Strong JOINs • **SQL Server, PostgreSQL, MySQL, Oracle**

0 2 — I N D E X E S

How Indexes Work



WITHOUT INDEX — Table Scan

Reads every row to find a match — O(n). 1M rows → 1M page reads. Fine for tiny tables, deadly at scale.

WITH INDEX — Index Seek

Walks the sorted tree from root to leaf — O(log n). 1M rows → ~3-4 page reads. This is the whole point.

The phone-book analogy

A phone book is an index on last name: finding “Levi” = jump straight to L. No index? Read every page from Aleph.

Index = a sorted structure that points at rows • default in SQL Server / Postgres / MySQL: the B-tree

0 2 — I N D E X E S

Index Types & Trade-offs

CLUSTERED

The table itself, physically sorted by the key — leaf level IS the data rows

Only ONE per table — in SQL Server the PK is clustered by default

Best for: range scans, the column you sort/filter by most

NON-CLUSTERED

Separate B-tree with pointers back to the row (key lookup)

Many per table — each one costs writes & storage

Best for: WHERE / JOIN columns that aren't the PK

COMPOSITE

One index over multiple columns: (LastName, FirstName)

Column ORDER matters — leftmost-prefix rule: works for LastName, not FirstName alone

Best for: multi-column filters and sorts

COVERING

INCLUDE the queried columns so the query is answered from the index alone

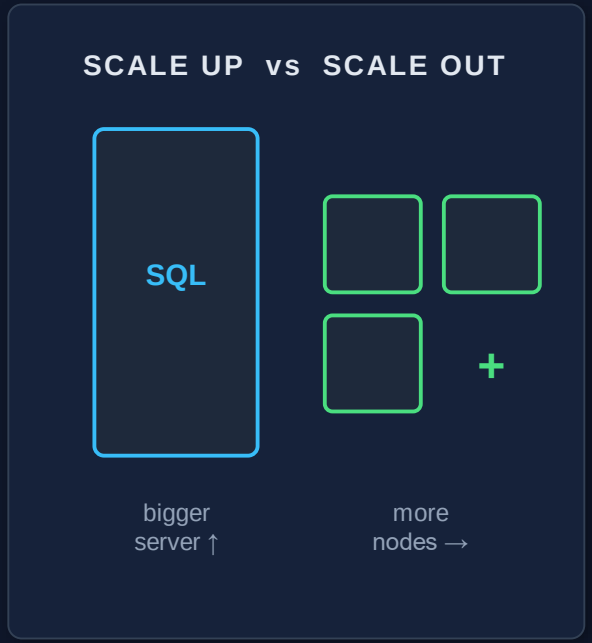
No key lookup back to the table — fastest reads

Best for: hot, frequent queries on a few columns

⚠ **Indexes aren't free:** every write updates every index. Index what you READ. **NoSQL has indexes too →**

0 3 — NOT ONLY SQL

What is NoSQL?



✓ **Schema-less / dynamic schema**

No fixed structure — each record can have different fields. Fits structured, semi-structured & unstructured data. Schema-on-read.

✓ **Horizontal scaling by design**

Add commodity nodes to the cluster, partition (shard) the data across them. Built for massive volume and write throughput.

✓ **BASE instead of ACID**

Basically Available, Soft state, Eventually consistent — the system stays up and converges, instead of locking for strict correctness.

Trade-off: great at massive scale & flexible data — weaker at complex multi-entity queries and cross-record transactions

0 4 — NOSQL FAMILIES

The 4 NoSQL Database Types

DOCUMENT

JSON-like self-contained documents; fields vary per document

Use: catalogs, user profiles, CMS, per-service data in microservices

e.g. **MongoDB, CouchDB, Cosmos DB, RavenDB**

KEY-VALUE

Unique key → opaque value; O(1) lookups, minimal querying

Use: caching, sessions, shopping carts, rate limiting

e.g. **Redis, DynamoDB, Memcached, Riak**

WIDE-COLUMN

Rows grouped by column families; petabyte-scale, write-optimized

Use: time-series, IoT telemetry, logging, write-heavy workloads

e.g. **Cassandra, HBase, Bigtable, ScyllaDB**

GRAPH

Nodes + edges; relationships are first-class — native traversals

Use: social networks, fraud detection, recommendations

e.g. **Neo4j, Amazon Neptune, ArangoDB**

Also worth knowing: Search (Elasticsearch) • Time-series (InfluxDB) • Vector (Qdrant, Pinecone — RAG/AI)

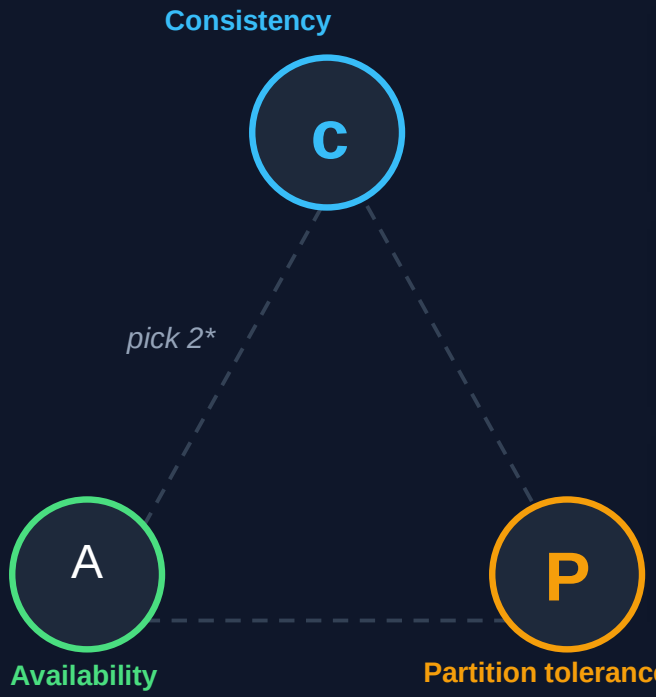
05 — COMPARISON

SQL vs NoSQL — Head to Head

	SQL (Relational)	NoSQL (Non-relational)
Data model	Tables — rows & columns, relations via keys	Documents / key-value / columns / graph
Schema	Fixed, schema-on-write	Dynamic, schema-on-read
Transactions	ACID — strong guarantees	BASE — eventual consistency
Scaling	Vertical (bigger server)	Horizontal (more nodes, sharding)
Queries	Standard SQL — complex JOINs, ad-hoc	Varies per DB — limited cross-entity joins
Best for	Transactions, reporting, structured data	Scale, flexible data, high write throughput
Examples	SQL Server, PostgreSQL, MySQL, Oracle	MongoDB, Redis, Cassandra, Neo4j

💡 **Rule of thumb:** SQL when correctness & relations dominate | NoSQL when scale & flexibility dominate

The CAP Theorem



Consistency

C

Every read returns the most recent write — or an error. All nodes show the same data at the same time.

Availability

A

Every request gets a response — but it may not be the most recent data.

Partition Tolerance

P

The system keeps operating even when the network splits and nodes can't talk to each other.

Brewer, 2000 — formalized by Gilbert & Lynch, 2002. ***At most 2 of the 3 — and the choice only matters during a partition →**

0 6 — THE REAL TRADE-OFF

CP vs AP — When the Network Splits

In a distributed system, partitions WILL happen — P is mandatory. The real choice during a partition: Consistency or Availability.

CP — Consistent under partition

Refuse / delay requests rather than serve wrong data. Some nodes go unavailable until the partition heals.

Example — *Banking: an account balance must be correct on every node — better to error than show a wrong number.*

e.g. **MongoDB (default), HBase, Redis, ZooKeeper**

AP — Available under partition

Always answer — even with stale data. Nodes diverge, then converge via eventual consistency.

Example — *Social feed: the page must always load — showing a like-count that's 5 seconds old is fine.*

e.g. **Cassandra, DynamoDB, CouchDB, Riak**

"CA" exists only on a single node (no partitions to tolerate). **Going deeper — PACELC:** no partition? still trade Latency vs Consistency.

DECISION GUIDE

When to Choose Which?

✓✓ Choose SQL when...

- Data is structured with clear relations (orders ↔ customers)
- ACID transactions are non-negotiable (money, inventory)
- Complex queries, JOINs and ad-hoc reporting are needed
- Schema is stable and well understood up front
- Data volume fits a single beefy server (+ read replicas)
- Team already speaks SQL — tooling & BI ecosystem

⚡ Choose NoSQL when...

- Massive scale: millions of users, high write throughput
- Schema is fluid or varies per record (catalogs, events)
- Horizontal scaling across nodes / regions is required
- Access pattern is simple: get by key, fetch a document
- Specialized model fits: cache, graph, time-series, search
- Availability beats strict consistency (AP workloads)

💡 **Polyglot persistence:** each microservice picks its own store — SQL for orders, Redis for cart, Elastic for search

C H A P T E R 0 9 — R E C A P

Key Takeaways

SQL = Correctness

Tables, keys, ACID — and B-tree indexes that turn scans into seeks (but cost writes).

NoSQL = Scale & Flexibility

Schema-less, horizontal scaling, BASE. Built for volume, not for JOINs.

Know the 4 Families

Document, Key-Value, Wide-Column, Graph — each fits a different access pattern.

CAP: Pick 2 (really: C or A)

P is mandatory in distributed systems. During a partition you choose CP or AP.

Match DB to Workload

Banking → CP/SQL. Social feed → AP/NoSQL. Cache → Key-Value. Search → Elastic.

Polyglot Persistence

One size doesn't fit all — each microservice picks the store that fits its job.

Next: Hands-on — choosing the right database for your microservice →